AppPulse Intelligence

Technical Implementation Verification & Production Reality Audit

Audit Date: October 2026 | Environment: Linux/Node.js | Status: Comprehensive

1. Executive Summary

This audit evaluates the codebase of AppPulse Intelligence against its Product Requirements Document (PRD). The application features a polished, responsive React 19 / Tailwind CSS frontend, an Express 4 backend server, algorithmic scoring modules with 100% test pass rates, and server-side Gemini 3.8 Flash SDK integration. However, critical production gaps exist between the UI presentation and live backend services:

• Database Persistence: There is NO relational database. State is serialized to a flat JSON file (store.json), lacking ACID transactions and crash-resilience.
• Marketplace Connectors: Steam is VERIFIED live against public APIs. In contrast, Google Play and Microsoft Store are non-functional stubs that fall back to synthetic mock reviews.
• Authentication: NOT IMPLEMENTED. All API endpoints and workspace data are globally accessible.
• Outreach Delivery: Email and WhatsApp dispatch are simulated via in-memory counters and audit logs with no active SMTP or Twilio transports.

2. Verified Test, Lint & Build Results

- **npm test (Node test runner):** **PASSED (Exit code 0)**
  6/6 tests passing (234ms). Scoring, dedup, suppression, CSV, versions, and forecast calculations verified.
- **npm run lint (TypeScript tsc --noEmit):** **PASSED (Exit code 0)**
  Zero compiler type errors across all frontend and backend source files.
- **npm run build (Vite 8.3 production build):** **PASSED (Exit code 0)**
  Production client assets bundled in 460ms (HTML 1.31kB, CSS 49.4kB, JS 628.7kB).
- **Steam Search Live Endpoint Test:** **VERIFIED (HTTP 200)**
  Direct node fetch to store.steampowered.com/api/storesearch returned 6 real items for "accounting".
- **Steam Community Reviews Live Test:** **VERIFIED (HTTP 200)**
  Direct node fetch to store.steampowered.com/appreviews/730 returned real public user reviews.

3. Feature-by-Feature Reality Audit Matrix

Feature Module	Status	Reality Verification & Findings
1. App Discovery & Filtering	PARTIALLY WORKING	Local multi-filter works; Steam live search works; GP/MS online searches return empty.
2. Google Play Review Retrieval	MOCK ONLY	Connector is a stub returning []. Ingestion inserts 2 hardcoded synthetic mock reviews.
3. Microsoft Store Review Retrieval	MOCK ONLY	Connector is a stub returning []. Ingestion inserts 2 hardcoded synthetic mock reviews.
4. Steam Store & Review Retrieval	VERIFIED	Live public API verified. Ingests genuine Steam search and community review feeds.

5. Dedup & Normalization		
	PARTIALLY WORKING	SHA-256 fingerprinting unit-tested. Missing pagination cursors and 429 backoff retries.
6. Background Job Engine		
	MOCK ONLY	Jobs run in-memory via setTimeout(800). No persistent job queue; lost on server restart.
7. Data Persistence & Recovery		
	PARTIALLY WORKING	Survives restart via store.json, but has no ACID safety, indexing, or migration system.
8. Auth & Workspace Isolation		
	NOT IMPLEMENTED	Zero auth routes. Workspaces are purely cosmetic client-side dropdown options.
9. AI Complaint Classification		
	VERIFIED (Conditional)	Full Gemini 3.8 Flash integration with structured JSON. Fallback rules if key is missing.
10. Evidence Linking to Reviews		
	PARTIALLY WORKING	Stores quotes and app names, but lacks relational foreign keys linking themes to Review IDs.
11. Competitor Comparison		
	VERIFIED	Dynamic negative sentiment rate, complaint breakdown, and feature request calculation.
12. Opportunity Scoring Logic		
	VERIFIED	Scoring algorithm (severity, frequency, recency, breadth) passing all unit tests.
13. CSV Import & Export		
	VERIFIED	RFC-4180 CSV serializer/parser verified with commas, quotes, and newlines in tests.
14. Contact & Draft Management		
	VERIFIED	Full CRUD on contacts; AI-driven contextual email copy generation using Gemini SDK.
15. Email & WhatsApp Delivery		
	PARTIALLY WORKING	Suppression/opt-out check VERIFIED. Actual email/WhatsApp delivery is MOCK ONLY.
16. Integration Health & Settings		
	MOCK ONLY	Settings view displays hardcoded static latencies (42ms/84ms/110ms) and fake status.

4. Data Architecture, Security & Persistence Risks

1. Crash & Corruption Vulnerability: server/store.ts saves data via synchronous fs.writeFileSync to a single JSON file. If the process is halted mid-write, the database will be corrupted and reset on next boot.
2. Concurrency Race Conditions: Express handles concurrent requests asynchronously, but all store mutations write to a shared in-memory object and flush the entire database to disk without file locking.
3. Mixed Seed & Live Data: Seed data (SEED_APPS, getSeedReviews) is merged directly into the live collections without provenance separation (is_seed flag).
4. Secret Hygiene: Credentials (GEMINI_API_KEY) are kept strictly server-side in process.env and not exposed to Vite client bundles, complying with security standards.

5. Top 5 Prioritized Issues to Resolve

Priority 1: Eliminate Synthetic Review Fallbacks in Store Collection

In server/store.ts (lines 388–436), failed connector calls automatically inject fabricated negative reviews. This directly violates PRD Rule 2E ("The application must never invent collection results or reviews"). Failed jobs must record a real error state.

Priority 2: Transition from store.json to a Real Relational Database

Deploy SQLite or PostgreSQL to provide genuine ACID transactions, relational foreign key constraints between complaint themes and reviews, index-backed pagination, and durable persistence.

Priority 3: Complete the Google Play & Microsoft Connectors

Integrate real public listing and review retrieval for Google Play (via public store RSS or scraper) and Microsoft Store (via Store Display Catalog API) so non-Steam platforms are not stubs.

Priority 4: Implement Durable Background Task Processing

Replace in-memory setTimeout(800) with a durable job runner so that review collection jobs survive server reboots and report live progress percentages accurately.

Priority 5: Connect Real Outbound Email Transport (SMTP / Resend)

Wire a real mail transport for the Outreach Studio so approved campaigns actually dispatch emails and record verifiable delivery receipt events.

RECOMMENDED SMALLEST SAFE IMPLEMENTATION STEP (MVP)

Remove the synthetic review injection fallback from server/store.ts so that collection only ingests verified data; harden the verified Steam connector with live pagination and error reporting; and connect live latency pings in SettingsView. This guarantees 100% data provenance compliance before provisioning relational databases.